

Flujo de Baja de Inscripciones y

Autenticación JWT

Documento de Exposición de Proyecto - Programación IV

Este documento detalla el ciclo de vida completo del flujo de baja (eliminación lógica) de una inscripción, desde que el usuario realiza la acción en el Frontend hasta su persistencia en la base de datos PostgreSQL en el Backend, detallando en qué punto y cómo interviene la seguridad mediante tokens JWT.

1. El Origen: Frontend (El Botón y la Petición HTTP)

Cuando el usuario confirma que desea dar de baja una inscripción en el panel de control:

1. Se captura el ID de la inscripción a eliminar y se hace clic en el modal. Esto dispara un evento en `inscripciones.js` que ejecuta:

```
const response = await apiFetch(`/api/v1/inscripciones/${inscripcionIdAEliminar}`, {  
  
  method: 'DELETE'  
  
});
```

2. El helper `apiFetch` (en `auth.js`):

- Busca en el almacenamiento local del navegador (`localStorage`) bajo la clave `fcad_jwt_token`.
- Construye las cabeceras de la petición HTTP agregando la firma de autorización:
`Authorization: Bearer <TOKEN>`.
- Envía la solicitud HTTP DELETE con destino al backend.

2. El Filtro: Backend Middleware (Autenticación JWT)

La petición llega al backend Express. En `app.js`, el enrutador de inscripciones está protegido mediante una "guarda":

```
app.use('/api/v1/inscripciones', authMiddleware, inscripcionesRouter);
```

Antes de que la petición toque cualquier controlador de inscripciones, se ejecuta obligatoriamente el **authMiddleware**:

1. **Lectura del Token:** Obtiene el encabezado `Authorization` de los headers de la petición. Si no existe o no inicia con el formato `Bearer`, corta el flujo respondiendo `401 Unauthorized`.
2. **Validación de Firma y Expiración:** Utiliza la librería `jsonwebtoken` mediante:

```
const payload = jwt.verify(token, process.env.JWT_SECRET);
```

Si la firma es inválida o expiró, se arroja un error y responde `401`.

3. **Propagación de Identidad:** Si es válido, inyecta los datos descifrados del token en la petición: `req.user = payload` (haciendo que el ID del usuario quede disponible en `req.user.id`). Luego llama a `next()`.

Concepto Clave JWT: El token no requiere consultar a la base de datos para autenticar al usuario.

El servidor simplemente verifica la firma criptográfica usando la clave secreta, lo cual hace al sistema "stateless" (sin estado) y altamente eficiente.

3. El Controlador: HTTP Handler

Una vez superada la autenticación y las validaciones de formato de parámetros, la petición coincide con la ruta `DELETE /:id` en `inscripcionesRoutes.js` y se ejecuta `deleteInscripcion` en `inscripcionesController.js`:

- Obtiene el ID de la inscripción de la URL: `req.params.id`.
- Obtiene de manera segura el ID del usuario autenticado (inyectado por el middleware): `req.user.id`.
- Delega el flujo llamando a la capa de servicios:

```
await inscripcionesService.deleteInscripcion(req.params.id, userId);
```

4. El Servicio: Capa de Lógica de Negocio

En `inscripcionesService.js`, la función `deleteInscripcion` coordina el negocio:

1. **Validación de Existencia:** Consulta al repositorio mediante `inscripcionesRepo.findById(id)`. Si no se encuentra la inscripción (o ya está inactiva), lanza un error con `statusCode = 404`.
2. **Procesamiento:** Llama al repositorio para persistir la baja:

```
await inscripcionesRepo.deleteById(id, userId);
```

5. El Repositorio: Acceso a Datos (Base de Datos)

En `inscripcionesRepository.js`, la función `deleteById` interactúa directamente con la base de datos PostgreSQL utilizando la conexión en la piscina (`pool.query`):

```
UPDATE inscripciones

SET id_inscripcion_estado = 2,

    id_usuario_modificacion = $2,

    fecha_hora_modificacion = NOW()

WHERE id_inscripcion = $1
```

- **Soft Delete (Borrado Lógico):** No se utiliza un comando `DELETE FROM`. En su lugar, se actualiza el estado a `2` (Inactiva/Baja). Esto permite preservar la integridad referencial y el historial de datos.
- **Auditoría:** Se almacena de forma segura el responsable de la baja en `id_usuario_modificacion` (ID recuperado del JWT) y la marca de tiempo en `fecha_hora_modificacion` (`NOW()`).

6. Retorno y Redirección del Frontend

- El repositorio actualiza la base de datos, el servicio y el controlador responden exitosamente al frontend con un código HTTP `200 OK`.
- En el cliente, si la respuesta es exitosa se refresca la lista. Si en cualquier petición se hubiera recibido un error `401`, el helper `apiFetch` limpia el `localStorage` (`removeToken()`) y redirige automáticamente al usuario al login (`index.html`).

Apéndice: ¿Por qué `validarId` no tiene `next()`?

En `inscripcionesRoutes.js` vemos la ruta:

```
router.delete('/:id', validarId, validate, deleteInscripcion);
```

`validarId` (definido en `validators.js`) no es un middleware común, sino un **arreglo de ValidationChains** de la librería `express-validator`:

```
export const validarId = [
  param('id').isInt({ min: 1 }).withMessage('El ID debe ser un número entero positivo'),
];
```

Express aplanar automáticamente los arreglos de middlewares y ejecuta cada regla secuencialmente, llamando a `next()` de forma interna por nosotros. Estas reglas guardan los resultados de validación dentro de `req`. Luego, el middleware personalizado `validate` lee los resultados de error acumulados mediante `validationResult(req)`, decidiendo si corta la petición con un error `400` o si ejecuta `next()` para dar paso al controlador.